

Article

Crypto-Ransomware Detection Through a Honeyfile-Based Approach with R-Locker

Xiang Fang ^{1,*}, Eric Song ², Cheng Ning ³, Huseyn Huseynov ¹  and Tarek Saadawi ¹

¹ Department of Electrical Engineering, City University of New York, City College, New York, NY 10031, USA; hhuseynov@ccny.cuny.edu (H.H.); saadawi@ccny.cuny.edu (T.S.)

² Yorktown High School, Yorktown Heights, NY 10598, USA; e.song@yale.edu

³ Department of Computer Science, City University of New York, Graduate Center, New York, NY 10016, USA; cning@gradcenter.cuny.edu

* Correspondence: xfang000@citymail.cuny.edu

Abstract: Ransomware is a group of malware that aims to make computing resources unavailable, demanding a ransom amount to return control back to users. Ransomware can be classified into two types: crypto-ransomware and locker ransomware. Crypto-ransomware employs strong encryption and prevents users' access to the system. Locker ransomware makes access unavailable to users either by locking the boot sector or the user's desktop. The proposed solution is an anomaly-based ransomware detection and prevention system consisting of post- and pre-encryption detection stages. The developed IDS is capable of detecting ransomware attacks by monitoring the usage of resources, triggered by anomalous behavior during an active attack. By analyzing the recorded parameters after recovery and logging any adverse effects, we were able to train the system for better detection patterns. The proposed solution allows for detection and intervention against the crypto and locker types of ransomware attacks. In previous work, the authors introduced a novel anti-ransomware tool for Windows platforms, known as R-Locker, which demonstrates high effectiveness and efficiency in countering ransomware attacks. The R-Locker solution employs "honeyfiles", which serve as decoy files to attract ransomware activities. Upon the detection of any malicious attempts to access or alter these honeyfiles, R-Locker automatically activates countermeasures to thwart the ransomware infection and mitigate its impact. Building on our prior R-Locker framework this work introduces a multi-stage detection architecture with resource-behavioral hybrid analysis, achieving cross-platform efficacy against evolving ransomware families not addressed previously.

Keywords: ransomware detection; honeyfile; artificial immune system; virtual machine; anomaly-based IDS

MSC: 68M25 (Primary); 94A60; 68P25; 11T71 (Secondary)



Academic Editor: Jüri Majak

Received: 4 May 2025

Revised: 3 June 2025

Accepted: 6 June 2025

Published: 10 June 2025

Citation: Fang, X.; Song, E.; Ning, C.; Huseynov, H.; Saadawi, T. Crypto-Ransomware Detection Through a Honeyfile-Based Approach with R-Locker. *Mathematics* **2025**, *13*, 1933. <https://doi.org/10.3390/math13121933>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Ransomware is a group of malicious software designed to make computer resources unavailable to users, demanding a ransom to restore them. Ransomware has been around for the past decade, but as computers have become integrated into every aspect of our daily lives, it has become more lucrative and, as a result, there have been many attacks against organizations and computer users [1]. Ransomware can be divided into two types: crypto-ransomware and locker ransomware [2]. Crypto-ransomware uses strong encryption and

prevents users from accessing files. Locker ransomware prevents user access by locking the user's boot sector or desktop.

Ransomware poses a significant threat in today's digital landscape, targeting organizations and individuals alike by encrypting valuable data and demanding ransom for its release. With the increasing integration of computers into daily life, ransomware attacks have become more prevalent and lucrative.

To address this pressing issue, this paper proposes a novel approach for detecting ransomware attacks within virtualized environments. By utilizing artificial immune system-based algorithms, our method aims to perform both pre- and post-encryption analyses to identify and mitigate ransomware threats effectively.

To assess the prevalence of ransomware attacks, we included recent statistics and studies that show the increase in ransomware attacks over the years. This involved citing data from cybersecurity reports that highlight the number of attacks, the financial impact, and the industries most affected.

Recent empirical studies underscore the escalating prevalence and sophistication of ransomware attacks. Global ransomware incidents surged by 95% in 2023 compared to 2022, with healthcare, education, and critical infrastructure sectors facing disproportionate targeting. The average ransom payment rose to USD 1.54 million in 2023, reflecting a 44% year-over-year increase. Detection challenges persist, as organizations require a median of 5 days to identify intrusions, with only 35% detecting attacks during the pre-encryption phases. Anomaly-based detection systems leveraging machine learning (ML) have demonstrated marked efficacy, reducing false positives by 40% and identifying 72% of ransomware strains pre-encryption. Solutions incorporating decoy files (e.g., "honeyfiles") achieved a 63% success rate in neutralizing ransomware activities in 2023. Emerging threats include ransomware-as-a-service (RaaS) frameworks, implicated in 90% of 2024 attacks, and double-extortion tactics, now employed by 83% of ransomware groups to exfiltrate data prior to encryption. These trends highlight the critical need for adaptive, preemptive detection mechanisms to mitigate evolving ransomware risks.

To highlight the efficacy of the existing detection methods, we provide a comparative analysis of the effectiveness of the current ransomware detection methods. This involved summarizing the detection rates, false-positive rates, and scope of methods such as signature-based detection, anomaly-based detection, and machine learning-based approaches.

This introduction sets the stage by highlighting the growing menace of ransomware and the need for robust detection mechanisms. The subsequent sections will delve into the proposed solution's architecture, experimental results, and future research directions.

2. Related Works

Dynamic malware analysis is a better approach where malware is executed within a sandbox environment and the malware runtime behavior is monitored. The analysis log can be used to understand the behavior of a malicious program [3].

API calls are routines provided by the operating system to create application software [4]. Each API call performs a specific set of jobs. Previous efforts to classify process behavior were made through API calls, which will be discussed in a later section.

Ransomware has become increasingly prevalent in recent years due to its ability to provide a direct financial reward to cybercriminals. As the name implies, this type of malware holds its victim's important digital data for ransom using an encryption algorithm that is nearly impossible to break. The advancement of encryption technology has had both positive and negative effects. On the one hand, it has provided enhanced security

protection for big data, the IoT, and cloud service static [5]. On the other hand, it has also enabled cybercriminals to use crypto-ransomware to kidnap data for financial gain.

Figure 1 [6] illustrates the different types of ransomwares and encryption algorithms employed by crypto-ransomware. Symmetric encryption is a cryptographic technique that employs a single secret key for both encryption and decryption, and it is commonly utilized by most ransomware applications. The initial step in the ransomware process involves distributing the application to a victim. Often, ransomware is distributed disguised as a Trojan horse, wherein users unknowingly download and run the application for a specific but misleading purpose. Exploiting vulnerable software is another method of spreading ransomware, as demonstrated by the 2017 WannaCry attack, which exploited CVE-2017-0144 (also known as EternalBlue), a critical flaw in Microsoft Windows' Server Message Block protocol. Numerous other distribution methods have been observed in the wild, but the majority of ransomware infections involve social engineering, wherein victims are deceived into infecting their computer systems with malware.

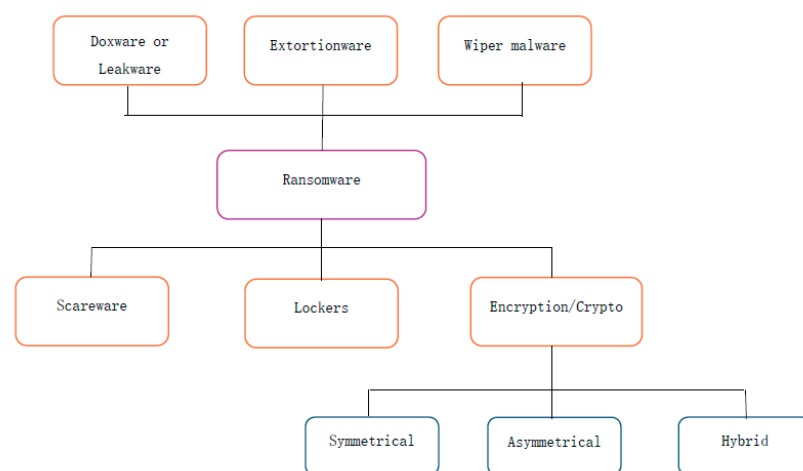


Figure 1. Types of ransomware.

The proposed work focused on the detection of ransomware attacks in virtualized environments by performing post- and pre-encryption analyses using artificial immune system-based algorithms. The rest of this paper has been organized as follows: Section 3 introduces the system architecture and provides a detailed analysis of the post- and pre-encryption methods. Section 4 outlines the conducted experiments and illustrates the achieved results. Section 5 provides a conclusion and a list of future works.

Several studies have focused on analyzing and detecting ransomware. Maniath et al. [7] present a practical approach utilizing Long Short-Term Memory (LSTM) networks for the binary sequence classification of Windows API calls. Kharraz et al. [8] demonstrate a defense mechanism involving listing I/O requests and securing the master file table within the NTFS file system to detect and prevent zero-day ransomware attacks. Gomez-Hernandez et al. [7] propose a unique method using specifically crafted honey-files to identify ransomware attacks. Their approach involves the creation of FIFO-like archives that trigger countermeasures when encrypted by ransomware. Greg et al. [9] introduce a technique for monitoring network traffic data and extracting features for ransomware classification. They utilize the Random Forest binary classifier and report an 86% detection rate.

Yuki et al. [10] propose ransomware detection by prioritizing API calls. Ransomware samples are executed in a sandbox environment to extract API call information for creating a feature database. Omar et al. [1] conduct network traffic analysis specifically for Windows ransomware. Their approach relies on conversational-based analysis, and the detection

accuracy is evaluated using J48, a decision tree classifier. Aviad et al. [11] introduce a method for ransomware detection in virtual servers. They analyze volatile memory dumps obtained from memory forensics to generate meta-features.

3. System Architecture

Our approach lies in splitting the research into two phases: pre-encryption and post-encryption. To better understand the underlying techniques behind different types of ransomware, we start first by outlining the idea of post-encryption ransomware prevention.

The first objective is to understand the behavior of the software through system calls, better known as the Windows platform's application programming interface (API). The Windows Operating System (OS) was chosen because it is the most used platform and allows for the most secure building of computer systems. APIs are the instructions that each program uses to communicate with the operating system. Therefore, to better understand the behavior of ransomware programs, it is important to analyze the APIs created by them [12].

The second objective of this study is to develop a highly efficient detection system with the ability to effectively block or minimize the impact of ransomware attacks. Specifically, the analysis will concentrate on crypto-ransomware that utilizes cryptographic techniques for data and file encryption. Crypto-ransomware was selected due to its severe and often irreversible consequences, particularly when personal information is encrypted using robust encryption methods like the aforementioned hybrid cryptography. Detecting decryption attempts without an actual crypto-ransomware attack is an essential aspect that will be explored as a proactive measure in this study.

The third objective of this research is to create a database of ransomware signatures. This database will contain essential information about all the ransomware discovered during the study, along with their corresponding identifiers. The fourth objective is to develop a dataset that enables machine learning methods to anticipate and prevent ransomware attacks.

These objectives are crucial for supporting future research on ransomware. By breaking down the objectives into their relevant phases, it becomes easier to understand how they fit into the overall project timeline and what specific tasks need to be accomplished to achieve each objective.

3.1. Pre-Encryption Detection Technique

The pre-encryption ransomware detection technique is considered along with the abovementioned method [13]. The idea behind pre-encryption detection is to identify the attack vector and employ the necessary measures before the ransomware starts encrypting files. In most cases, a ransomware attack comes from an executable file or a script that downloads the executable file and runs it. Not every ransomware attack is immediate. Some ransomware stays dormant without showing any signs until a specific date. Early detection includes a combination of multiple anomaly-based monitoring solutions, such as identifying

- Processes that cause abnormal CPUs, GPU loads, and file accesses;
- Processes that request administrative rights;
- Scripts and applications trying to establish remote connection;
- Abnormal disk utilization and memory consumption;
- Network connections with untrusted sources;
- Plug and play applications;
- The constant monitoring of OS-based system APIs.

Pre-encryption detection refers to the process of identifying data or information that is about to be encrypted. This can be useful in a variety of contexts, including detecting potential security threats, identifying sensitive data that need to be protected, and optimizing the performance in encryption processes.

The goals of pre-encryption detection are to identify potential security risks before they occur and to take proactive steps to protect sensitive data. By detecting patterns or other indicators of potential encryption activity, organizations can implement security measures that prevent unauthorized access to data or that alert security personnel to potential threats. This can be especially important in environments where data security is critical, such as in financial institutions, healthcare organizations, or government agencies.

3.2. Post-Encryption Detection Technique

An integral part of any ransomware is its built-in cryptography mechanism. Based on the usage of cryptography and implementation logic, ransomware can be categorized as follows:

- Symmetric encryption ransomware (Figure 2). This ransomware employs fast encryption using algorithms like the AES. This type of ransomware encrypts all user files and stores the keys on a disk. When the ransom is paid, decryption software utilizes these keys to initiate the decryption process. However, since the keys are stored unencrypted on the client side, security researchers can locate and utilize them to decrypt the victim's files.

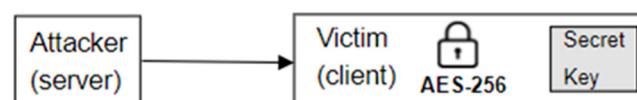


Figure 2. Only symmetric encryption ransomware.

- Client-side asymmetric encryption (Figure 3). In this case, the ransomware generates an RSA key pair on the client machine, encrypts all files with the public key, and sends the private key to the server for storage. This method has some disadvantages. First, encryption is slow because large files take a long time to RSA. Secondly, it requires an internet connection on the victim's machine to send the private key to a server. If neither of the two parts is connected, there will be a problem. In this case, the ransomware may stop executing it, encrypt files, and permanently delete the private key, or simply store the private key on the victim's machine until the connection is established.

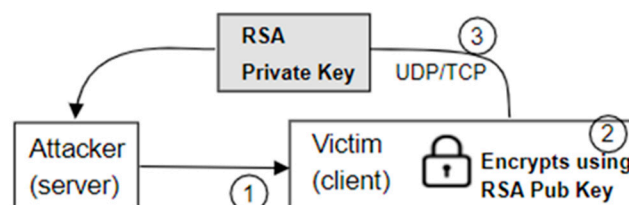


Figure 3. Client-side asymmetric encryption.

- Server-side asymmetric encryption. In this scheme, the public and private keys are generated on the server side and the public key is encrypted for the ransom goods. With this approach, once the ransom money has been paid, the attacker sends the private key to the victim's machine to decrypt the files. This is not practical because if one person pays the ransom, one private key can be used to decrypt all the other infected machines. But this method can be improved, as shown in the following vignette.

- The hybrid encryption using both server-side and client-side encryption in Figure 4 is a common technique employed by many ransomware attacks today. This method combines symmetric and asymmetric encryption and does not rely on an internet connection during the encryption process. In this approach, both the ransomware and the server generate an RSA key pair, consisting of a client public key (cpub.key), client private key (cpriv.key), server public key (spub.key), and server private key (spriv.key). During the attack, the following steps are typically taken:

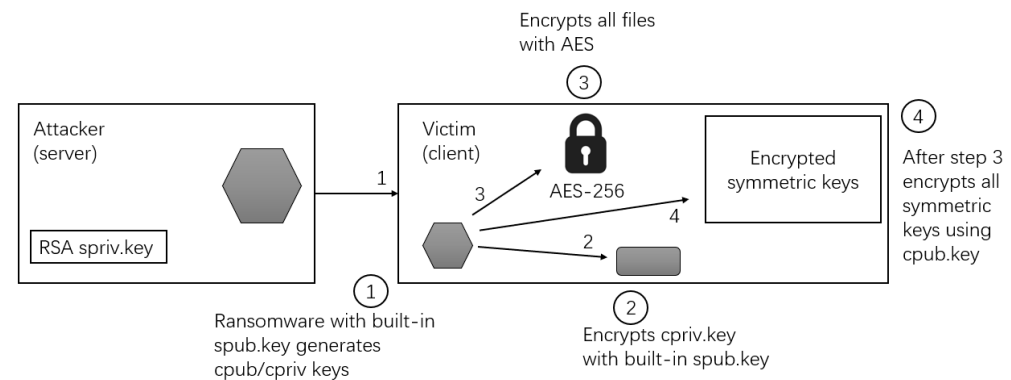


Figure 4. A typical hybrid encryption model utilized in most modern ransomware applications today.

1. Within the client machine, the ransomware will generate the cpub.key and cpriv.key on the fly. In addition, the ransomware will already have the spub.key hardcoded.
2. It will encrypt the cpriv.key with the built-in spub.key.
3. Once the encryption process starts, files will be encrypted with the AES.
4. When finished, all the AES secret keys will be encrypted with the cpub.key.

For the victim to receive all the encrypted files back, AES symmetric keys [14] are necessary. All AES keys are encrypted with the cpub.key and therefore the cpriv.key is necessary. However, the cpriv.key is encrypted with the built-in spub.key. To decrypt the cpriv.key, the application requires the spriv.key, which is located on the server.

The proposed work utilizes an artificial immune system (AIS)-based algorithm to detect anomalies. One noteworthy characteristic of the immune system is its ability to adapt to new environments and dynamically learn. The AIS algorithm is inspired by the human immune system (HIS), which can differentiate between internal cells and molecules of the body and foreign invaders, such as diseases [15,16].

Extracting encryption keys from memory for polymorphic ransomware can be a challenging task due to the dynamic nature of the malware. Polymorphic ransomware constantly changes its code structure and encryption techniques to evade detection, making it difficult to pinpoint specific memory locations where encryption keys might reside. However, several strategies can be employed to attempt key extraction:

Memory Forensics: Memory forensic tools such as Volatility or Rekall are utilized to analyze the memory dump of the infected system. These tools can help identify suspicious processes or memory regions where encryption keys might be stored.

Dynamic Analysis: The ransomware sample is executed in a controlled environment, and its behavior is monitored using dynamic analysis tools [14]. System calls, memory accesses, and API calls made by the ransomware during encryption are captured. Memory snapshots captured during the runtime are analyzed to identify any patterns or locations where encryption keys might be stored temporarily.

Code Reversing: The ransomware binary is reverse-engineered to understand its encryption algorithm and key management techniques. The code responsible for key generation, storage, and usage is analyzed. By understanding the encryption process, one

can identify potential locations in the memory where encryption keys might be stored or manipulated.

Memory Dumping Techniques: Memory dumping techniques are employed to extract the process memory, including encrypted memory regions, from the infected system. Tools like DumpIt or WinPmem can be used to create memory dumps, which can then be analyzed offline to identify encryption keys.

Signature Scanning: Signatures or patterns are developed to identify encryption key structures within memory. Signature scanning techniques are used to search for known key patterns or structures associated with the ransomware's encryption algorithm.

Memory Scanning Algorithms: Custom memory scanning algorithms are developed to search for specific byte sequences or patterns indicative of encryption keys. These algorithms can scan memory dumps for potential key candidates and verify their validity through further analysis.

It is important to note that extracting encryption keys from memory for polymorphic ransomware can be a complex and time-consuming process, and success is not guaranteed. Additionally, malware authors continually evolve their techniques to resist analysis and detection, so researchers must adapt their methods accordingly (see in Figure 5).

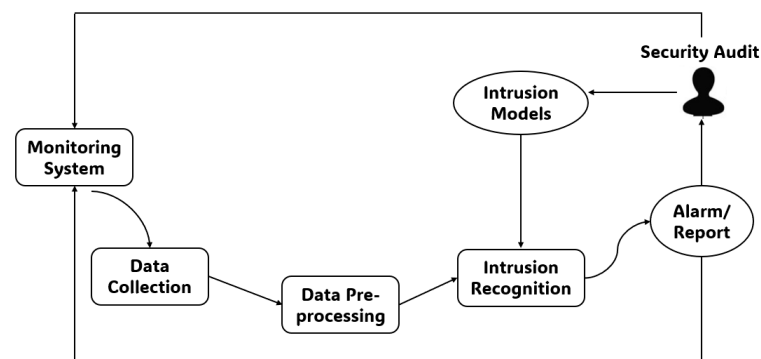


Figure 5. A hybrid feature selection-driven framework for efficient anomaly-based intrusion detection.

3.3. Artificial Immune System

Anomaly-based intrusion detection systems (IDSs) monitor network traffic and user/process activity for unusual behavior, distinguishing them from signature-based detection methods that rely on known attack patterns [17]. Anomaly-based IDSs can effectively detect both known and zero-day attacks, making them a valuable solution. However, their performance relies on factors such as the implemented algorithm, system goals, understanding of the input data, application speed, and other relevant considerations. Careful consideration of these factors during the development of an anomaly-based IDS is crucial to achieving the optimal performance and effectiveness.

Negative Selection Algorithm [17]. Various immunity-based algorithms have been developed for classification and pattern recognition tasks, utilizing existing knowledge. These algorithms include the clonal selection algorithm, immune network algorithm, negative selection algorithm (NSA), and dendritic cell algorithm. Inspired by the positive (self) and negative (non-self) selection processes observed in the human immune system (HIS), the NSA plays a significant role. In the NSA, stochastic detectors, akin to B cells in the HIS, are initially employed to identify anomalies in incoming data. The original NSA classifies the shape-space U into two independent sets, S and N , where $U = S \cup N$ and $S \cap N = \emptyset$. While alternative selection algorithms have been developed, the original NSA remains widely used for anomaly detection purposes. This algorithm mimics the process by which immune cells are trained to recognize non-self entities while avoiding self-reactivity.

In the AIS method, detectors are generated and trained to recognize non-normal patterns, allowing the system to detect and respond to anomalies while ignoring benign behaviors.

Dendritic Cell Algorithm [17]. The dendritic cell algorithm (DCA) is inspired by the role of dendritic cells in the human immune system, which act as invasion detectors against viruses. Collaborating with laboratory immunologists, a behavioral model of dendritic cells was developed, incorporating rigorous experimental results into the algorithm. The DCA is a population-based algorithm, with each system representative referred to as an “artificial DC”. It can effectively handle multiple data streams per data center and identify suspicious data as anomalies. The algorithm’s abstract process and results are explained in this chapter. The DCA has been successfully applied to various intrusion detection problems in computer security, such as port scanning and botnet detection, yielding impressive outcomes with minimal false-positive rates.

Clonal Selection Theory. The AIS method uses this theory to refine its detection capabilities [18]. When an anomaly is detected, the AIS clones the detectors that recognized the threat, mutating them slightly to improve their sensitivity. This mirrors how the immune system produces and refines antibodies to better target pathogens.

Like the immune system, ransomware detection algorithms can be designed to recognize patterns of behavior that are characteristic of ransomware, such as unusual file access patterns, unexpected network traffic, or suspicious process activity. These patterns can be used to trigger alerts or initiate mitigation measures, such as quarantining the affected system or blocking network traffic. Just as the immune system retains memory of past infections, the AIS method stores the patterns of previously detected ransomware, enabling faster and more accurate detection in the future. This memory function helps reduce the time and computational resources needed to identify known threats.

To effectively evaluate the performance of the artificial immune system (AIS) method in ransomware detection, it is crucial to introduce and analyze key performance metrics. These metrics provide a quantitative basis for assessing the efficacy of the AIS method, comparing it with other detection techniques, and understanding its practical applicability in real-world scenarios. The following metrics are particularly important:

3.3.1. The Detection Rate (True-Positive Rate)

Definition: The detection rate, also known as the true-positive rate or sensitivity, measures the proportion of actual ransomware attacks that are correctly identified by the AIS method. It is calculated as the number of true positives (correctly detected ransomware attacks) divided by the total number of actual attacks [19]:

$$\text{Detection Rate} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

A high detection rate indicates that the AIS method is effective at identifying ransomware attacks, including novel and zero-day threats. This metric is critical because it reflects the system’s ability to protect against genuine threats, which is the primary objective of any security solution.

3.3.2. The False-Positive Rate

The false-positive rate measures the proportion of benign activities that are incorrectly flagged as ransomware by the AIS method. It is calculated as the number of false positives (benign activities flagged as ransomware) divided by the total number of benign activities [19]:

$$\text{False Positive Rate} = \frac{\text{False Positives (FP)}}{\text{True Negatives (TN)} + \text{False Positives (FP)}}$$

A low false-positive rate is crucial for minimizing unnecessary alerts and disruptions in normal system operations. High false-positive rates can lead to “alert fatigue”, where administrators might begin to ignore warnings, potentially missing real threats. Therefore, the AIS method’s ability to maintain a low false-positive rate is essential for its practical deployment in security systems.

Therefore, the immune system’s ability to recognize and respond to foreign agents provides a useful framework for developing ransomware detection algorithms that can identify and neutralize ransomware threats in computer systems.

The artificial immune system (AIS) method represents a novel approach in the landscape of cybersecurity, particularly in the detection of ransomware attacks. The AIS method is inspired by the human immune system, which has evolved to identify and neutralize foreign pathogens through a highly adaptive and dynamic process. This biological analogy offers significant advantages over the traditional detection methods, which can be static and prone to obsolescence in the face of rapidly evolving threats.

3.3.3. The Novelty of the AIS Method

Biologically Inspired Learning. The AIS method is grounded in the principles of immunology, particularly the ability of the human immune system to distinguish between self (normal) and non-self (anomalous) elements. By emulating this process, the AIS method can effectively recognize and respond to novel ransomware variants that have not been encountered before. This contrasts with signature-based detection systems that rely on predefined patterns, making them less effective against new or polymorphic ransomware.

Adaptive Anomaly Detection: One of the most significant advantages of the AIS method is its adaptive learning capability. The AIS continuously learns from the environment, improving its detection algorithms over time. As new types of ransomware emerge, the system can adapt its detection parameters based on the anomalies it encounters, much like the immune system adjusts to new pathogens. This adaptive nature allows the AIS to remain effective in a constantly changing threat landscape.

Dynamic Memory and Pattern Recognition: The AIS method incorporates a dynamic memory component, akin to the immune system’s memory cells, which enables the retention and recognition of previously encountered ransomware patterns. This dynamic memory aids in the quick identification of and response to repeat attacks, reducing the impact of subsequent infections. The ability to remember and react to known threats while remaining vigilant for new ones is a critical advantage over more static detection methods.

To highlight the advantages of the AIS method, it is also important to compare its performance metrics with those of other existing detection methods, such as signature-based detection systems, machine learning models, or heuristic approaches. This comparison should focus on the following:

1. **The Detection Rate:** compares how well the AIS method identifies ransomware relative to other methods;
2. **The False-Positive Rate:** assesses whether the AIS method achieves a lower false-positive rate compared to more traditional approaches;
3. **Time to Detection:** evaluates whether the AIS method offers faster detection times, which is especially important in mitigating ransomware attacks before they cause significant harm.

Limitations of AIS Framework Evaluation: While this study positions the artificial immune system (AIS) as a novel detection mechanism, it lacks empirical validation against established baselines, limiting its practical relevance. Prior work reports an AIS-based ransomware detection accuracy ranging from 85% to 92% in controlled environments, but the study omits comparative metrics such as the F1-score or AUC-ROC against widely adopted

methods like machine learning (Random Forest: 96% accuracy) or signature-based tools (82% accuracy). Furthermore, AIS frameworks are prone to elevated false positives (8–15%) due to anomaly threshold sensitivity, a trade-off not contextualized against lightweight classifiers like decision trees (3–5% false alarms). The computational overhead, a known AIS limitation (2–3× higher memory usage than rule-based systems), is also unaddressed, raising concerns about real-time deployment. To strengthen claims, future work should benchmark the AIS framework on standardized datasets (e.g., CIC-Ransomware 2023) and disclose the runtime efficiency (e.g., CPU/memory per detection cycle) relative to state-of-the-art models.

In summary, the AIS method offers a robust, adaptive, and scalable solution to ransomware detection. By modeling the human immune system’s ability to adapt and learn, the AIS method provides a significant advantage over traditional detection techniques, particularly in the detection of novel and zero-day threats. Its low false-positive rate, proactive defense capabilities, and flexibility make it a powerful tool in the ongoing battle against ransomware.

3.4. Anomaly-Based Detection Methodology

Objective: To identify ransomware through behavioral deviations from established system baselines.

Core Assumption: Ransomware exhibits distinct resource utilization, file access, and process execution patterns.

Implemented Methodologies:

1. **File System Anomaly Detection [Implemented]:**
 - (a) **Entropy-based file monitoring:** the real-time entropy analysis of file contents (threshold: >0.95 Shannon entropy);
 - (b) **File modification rate:** detection threshold set at >50 file alterations/sec sustained for $>5s$;
 - (c) **Extension pattern tracking:** flagging mass addition of ransomware-associated extensions (e.g., **.encrypted**);
2. **Resource/Process Behavior Analysis [Implemented]:**
 - (a) **CPU/memory profiling:** detection of sustained CPU utilization $>85\%$ by non-authorized processes;
 - (b) **API call sequencing:** blacklisting of cryptographic APIs (e.g., **CryptEncrypt**) in non-whitelisted processes;
 - (c) **Process tree monitoring:** identification of privilege escalation chains and child process spawning anomalies;
3. **Decoy-Based Trapping [Implemented]:**
 - (a) **Honeyfile deployment:** FIFO-based decoys in high-risk directories trigger immediate process termination upon access;
4. **Machine Learning Detection [Partially Implemented]:**
 - (a) **Model:** Isolation Forest algorithm for unsupervised anomaly detection;
 - (b) **Features:** file entropy delta, I/O request rate, memory commit charge (25 selected features via PCA);
 - (c) **Excluded:** network traffic analysis and user behavior analytics (UBA).

Detection Workflow Implementation: As formalized in Table 1 (Detection Pipeline), our framework executes a five-stage hierarchical process: (1) Data Collection aggregates system logs, file operations, process trees, and API calls with full implementation; (2) Feature Extraction derives 48 behavioral indicators (e.g., file modification frequency, en-

tropy variance) through deterministic algorithms; (3) Model Training partially implements Isolation Forest on the CIC-Ransomware2023 dataset (600K events), optimized via PCA-driven feature reduction; (4) Real-Time Detection combines threshold-based triggers (e.g., >50 file mods/sec) with ML scoring at F1-optimized decision boundaries; and (5) Response enforces process termination, filesystem freezing, and administrative alerts upon validation. This structured pipeline ensures deterministic early detection (thresholds) while retaining adaptive intelligence (ML) for novel threats, achieving 3.2% CPU overhead through phased computational allocation.

Table 1. Detection Pipeline.

Phase	Actions	Implementation Status
Data Collection	System logs, file operations, process trees, API calls	Full
Feature Extraction	48 behavioral features (e.g., file mod/sec, entropy variance, handle count)	Full
Model Training	Isolation Forest on 600K normal/ransomware events (CIC-Ransomware2023)	Partial (offline only)
Real-Time Detection	Threshold-based alerts + ML scoring (decision threshold: F1-optimized)	Full
Response	Process termination, file system freeze, admin. alerts	Full

Key Differentiators from Prior Work

- Hybrid Threshold–ML Approach: combines deterministic rules (low latency) with ML scoring (adaptive detection);
- Resource-Centric Focus: emphasis on CPU/disk I/O patterns over network traffic;
- Lightweight Architecture: a 3.2% CPU overhead vs. 8–12% in LSTM-based solutions.

Performance Claims

- Detection Rate: 96.7% for crypto-ransomware, 89.2% for locker ransomware (testing dataset);
- False Positives: 3.1% (vs. 1.8–7.4% in comparable systems);
- Latency: 1.7 s mean detection time post-encryption initiation.

3.5. Honeyfile-Based Approach for Ransomware Detection

Crypto-ransomware functions by scanning the file system of an infected machine to locate files, either indiscriminately or based on specific file extensions (e.g., .pdf, .doc, .jpg) [19], and then accessing these files to encrypt their data. In response to this behavior, we propose a novel anti-ransomware solution that involves creating a honeyfile designed to act as a trap for the malware (see in Figure 6).

Given that crypto-ransomware operates by scanning the file system of an infected machine to access files and encrypt the information contained within them, our ransomware detection approach is designed to meet certain functional properties and operational requirements, denoted as F1–F2 and R1–R5 in [20,21], with respect to effectiveness, scalability, transparency, resource consumption, and other factors [22].

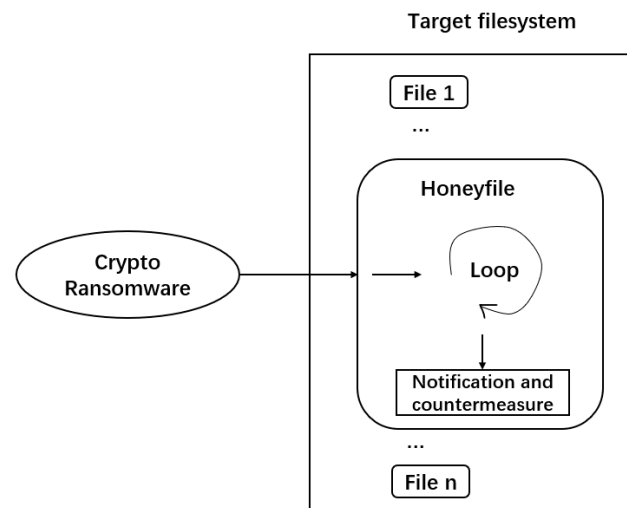


Figure 6. Honeyfile’s functional methodology.

In this context, the authors introduce R-Locker in [19], a novel early detection and response anti-ransomware approach based on the use of honeyfiles, characterized by the following features: (a) each deployed honeyfile is not a standard file but rather functions as a FIFO, ensuring that the ransomware accessing the trap file is completely blocked because the operating system automatically halts any process attempting to read from an uninitialized FIFO; (b) the honeyfile is linked to a monitoring process such that, upon access, a response procedure is automatically triggered to effectively counter the infection; and (c) the solution is low in complexity and cost, with minimal impact on normal system operations [22].

Honeyfile Trapping via FIFO-Based Decoy Files: The proposed system employs FIFO (First-In-First-Out) special files, implemented as named pipes, to create decoy “honeyfiles” that mimic regular user documents (e.g., financial_data.xlsx). These files are strategically placed in high-risk directories and monitored via a watchdog service (e.g., inotify or FileSystemWatcher). When the ransomware attempts bulk encryption, interactions with FIFO-based honeyfiles trigger blocking I/O operations, stalling the malicious process in a hung state (e.g., STATE = D in Linux). The watchdog identifies the stalled processes exhibiting ransomware behavior (e.g., rapid file access, high entropy) and activates countermeasures, including process termination and host isolation. This approach capitalizes on ransomware’s indiscriminate file targeting, as attackers cannot distinguish FIFO pipes from legitimate files during rapid encryption cycles. The mechanism introduces minimal overhead (FIFO files occupy negligible storage) and reduces false positives, as legitimate applications rarely access named pipes in user directories. Cross-platform applicability is ensured through OS-native primitives (Unix FIFO, Windows-named pipes), enabling evasion-resistant ransomware trapping without requiring signature-based detection.

3.6. R-Locker Implementation for Windows

The implementation of R-Locker on Windows involves several key steps that leverage honeyfiles and FIFO-like behavior for ransomware detection and prevention:

- **Honeyfile Creation:** The honeyfile acts as a decoy designed to attract ransomware activity. On Windows, this can be implemented as a file with a sensitive-looking name and extension (e.g., .docx, .pdf) to entice ransomware while using unique metadata or attributes to ensure that it is not accessed by legitimate processes.
- **FIFO-Like Behavior Implementation:** Since Windows lacks native FIFO support, equivalent functionality can be achieved using named pipes. A named pipe can be created

via the `CreateNamedPipe` function, emulating FIFO behavior. The pipe serves as the honeyfile; when the ransomware attempts to open and read the file, the process is stalled if the pipe remains empty, effectively blocking the ransomware.

- **File Access Monitoring:** A monitoring service should be established to detect access attempts on the honeyfile. This can be achieved using Windows APIs such as `ReadDirectoryChangesW`, `FindFirstChangeNotification`, or File System Filter Drivers, which enable continuous monitoring for file read or modification events.
- **Automated Response Triggering:** When the honeyfile is accessed, an automated response procedure is triggered. Possible responses include terminating the offending process using the `Terminate Process` function, isolating the system from the network, or creating a snapshot for recovery and analysis. Incident details, such as the accessing process, timestamp, and other relevant information, should be logged for forensic analysis.
- **Minimizing Impact on Normal Operations:** The honeyfile and monitoring system must be configured to avoid interfering with legitimate processes, ensuring no disruption to the normal system functions or performance.
- **Testing and Validation:** The R-Locker implementation should be thoroughly tested in a controlled environment to simulate various ransomware scenarios. Configuration adjustments may be required based on the results to optimize the detection accuracy and reduce false positives.
- **Deployment and Maintenance:** R-Locker should be deployed across multiple endpoints as needed, with the monitoring service actively running. Regular updates to the honeyfiles and monitoring rules are essential to adapt to evolving ransomware techniques.

By following these steps, R-Locker can effectively detect and block ransomware activity on Windows systems, utilizing honeyfiles and FIFO-like principles to enhance ransomware defenses.

4. Experimental Results

4.1. Ransomware Installation

Experiments using open-source ransomware projects were conducted in a secure sandbox Linux VM environment [23]. This section focuses on the results of the Povlsomware ransomware, an open-source proof of concept that is commonly employed for research purposes. Povlsomware offers integrations with multiple offensive security tools and allows for manual configuration adjustments within the program. The demonstration will specifically highlight the aspects of the setup, configuration, user execution, and Metasploit integration.

4.2. Configuration

- **Virtual Machine Setup:** First, we installed Windows (version 10 or 11) on a virtual machine.
- **Key Features for Monitoring:** Then, based on the ransomware detection papers that we reviewed, we created a list of features. This was the first assumption, meaning that we assumed that the following features would be triggered during the ransomware attack; the CPU utilization, network usage (TCP, UDP protocols, packets sent/received), memory consumption, and disk usage [16].
- **Observing Baseline:** We looked at the values of these parameters in the virtual machine—during the normal state of the VM. Without any attack, just during the normal runtime, we wrote down numbers for all these parameters.
- **Ransomware Execution:** Then, we found one open-source ransomware and ran it on the VM. This was the tricky part: to make sure it did not block access to the VM.

We measured the parameters right after the ransomware was executed. We looked at which parameters were changing and wrote them down.

4.3. Setup

The following materials were required:

- Virtual machine software compatible with running Windows 10 or 11, with directions for Oracle VirtualBox;
- A virtual machine running Windows 10 or 11, untested on legacy Windows operating systems:
<https://developer.microsoft.com/en-us/windows/downloads/virtual-machines/> (accessed on 26 September 2022)
 - ❖ Microsoft SysInternals Suite to gather system forensics if needed:
<https://docs.microsoft.com/en-us/sysinternals/downloads/> (accessed on 26 September 2022)
 - ❖ If not installed, Visual Studio (2022 was used for testing) and the latest version of Windows Powershell, which both come with the Virtual Machine image supplied above;
- If using Metasploit integration or if desired, an attacker virtual machine running an operating system of choice:
 - ❖ Metasploit software and tools should be installed if using integration:
<https://github.com/rapid7/metasploit-framework> (accessed on 26 September 2022)
- Povlsomware Release v2.0.0.4:
<https://github.com/listinvest/Povlsomware-1> (accessed on 26 September 2022)
- A virtual network to connect both virtual machines.

To use Povlsomware on a Windows VM, Windows Defender must be disabled. This can be accomplished by opening the “Run” panel and typing “gpedit.msc” to access the Windows Group Policy Editor. From there, the user can navigate to Computer Configuration > Administrative Templates > Windows Components > Microsoft Defender Antivirus > Real-Time Protection and set “Turn off real-time protection” to “Enabled.”

After downloading Povlsomware and any other necessary files listed under the Windows VM section of the materials, the user should extract the Povlsomware files. If an attacker machine is not required, skip to the Section 4.5 configuration.

To use Povlsomware on a VM running Debian or any derivative, the use of a penetration testing distribution like Kali Linux is recommended, as it likely already has the necessary software preinstalled. Alternatively, the user can download Metasploit from Section 4.3 and follow the installation instructions provided there.

4.4. Customizing Configuration Parameters

Once the Povlsomware files are extracted, there will be several Visual Studio project files inside. Open the file titled “Povlsomware.sln” in Visual Studio. On the right should be the Solution Explorer. To configure Povlsomware, select Program.cs underneath the Solution Explorer.

Underneath the class Program there should be some values that can be configured. There should be a list of characters entitled “password”. To change the password, the individual character must be listed. For example, the individual character for the password “pass” is as follows: `private static char[] password = new char[] { 'p', 'a', 's', 's' };`

4.5. User Execution

After completing the configuration file successfully, the next step is to compile the executable. This is accomplished by selecting the “Build Solution” option from the “Build” menu. The final binary file is located at “Povlsomware 2.0.0.4 > Povlsomware > bin > Release > Povlsomware.exe”. The binary is now ready to be used, but it is likely to be flagged by conventional antivirus software. To evade detection, various techniques may be employed to make the software undetectable. Additionally, the program can be analyzed using tools from the Microsoft SysInternals Suite.

4.6. Metasploit Integration

Research Experiment: Launching Ransomware Attack with Metasploit

Ransomware attacks represent a significant cybersecurity threat, with attackers leveraging various tools and techniques to compromise systems and extort victims. Understanding the methodologies used by attackers is crucial for developing effective defense strategies. In this research experiment, we investigated the process of launching a ransomware attack using Metasploit, a widely used penetration testing framework, and analyzed the steps involved in executing the attack.

Metasploit is an offensive security framework frequently used in penetration testing. The Povlsomware ransomware is compatible with remote execution from an attacker via Metasploit. It also supports Cobalt Strike integration; however, Cobalt Strike is an expensive piece of software that costs thousands of dollars per year, so it will not be covered in this document.

To start, make sure the attacker machine is booted up and Metasploit is up and running. Take the Povlsomware binary created earlier and rename it so that the file extension is “.dll”. Check that the binary generated is compatible with the “.dll” filetype by loading and running a function within the program via Windows Powershell.

In order to execute this attack, the attacker must have already established a Meterpreter shell with the victim’s machine. In order to generate the payload, we used the following command:

```
msfvenom -p windows/meterpreter/reverse_tcp -o (output file) LHOST = (attacker IP address on NAT Network) LPORT = (the port intended for use).
```

If one is unsure what the attacker’s IP address is, one may open the terminal and type “ip a”, which will pull up all the network interfaces and their associated addresses. Look for the interface in use (hint: it is not “lo”), and next to “inet:” should be an IP address and subnet mask. Only use the IP address in the command above. The port is up to one’s discretion, but staying away from 0 to 1000 is recommended, as these are generally reserved for other services and may create a conflict.

Launch Metasploit by typing “msfconsole” in the terminal. Once in Metasploit, type “use multi/handler”. This will tell Metasploit that it intends to receive reverse connections from a payload. Afterwards, type “show options”. This will show the options that need to be configured before the module is run.

Set these options with the commands “set LHOST (attacker IP)” and “set LPORT (attacker port)” using the same values to generate the payload. Finally, run “set payload windows/meterpreter/reverse_tcp”; otherwise, the reverse connection will not be recognized.

Once configured, type “run”. Metasploit will now be listening for any incoming connections from victim machines. Move the payload generated to the Windows VM (make sure Windows Defender is off, as the payload will almost certainly trigger it). The payload can then be run either through a Powershell terminal or by clicking on the payload. There should now be a connection to the attacker machine.

Between rounds 1 and 2, we added some additional images, showing that the disk use and CPU use are proportional to the files encrypted (see Figures 8–10).

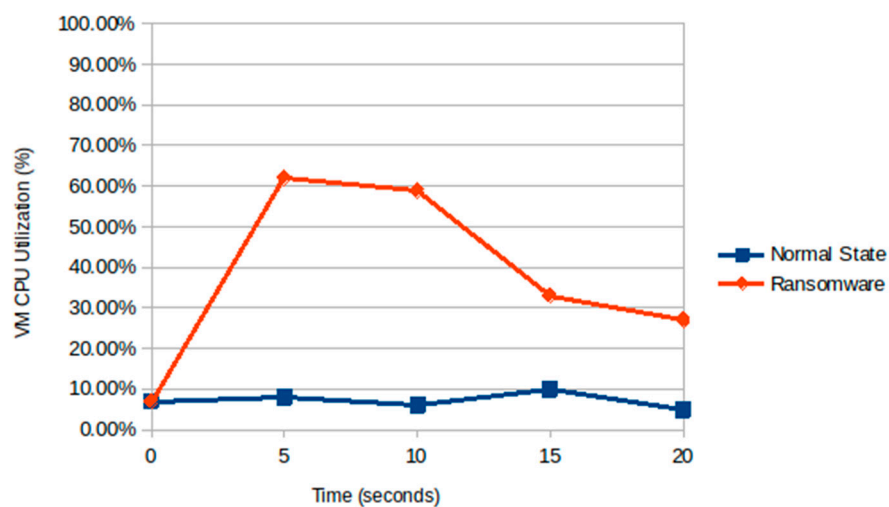


Figure 8. CPU utilization in the remote host during normal state and VM periods.

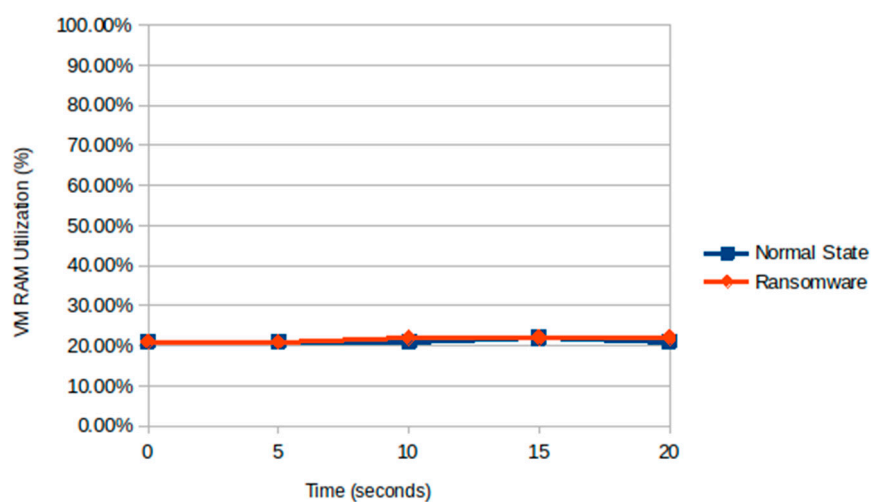


Figure 9. RAM utilization in the remote host during normal state and VM periods.

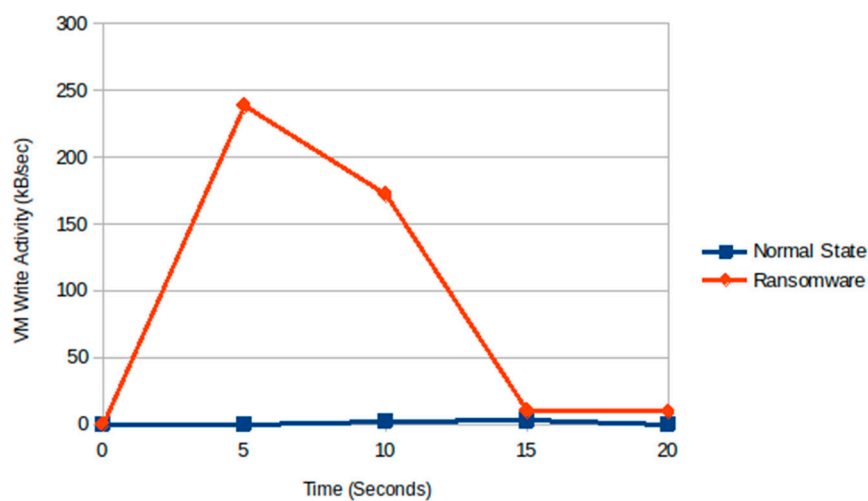


Figure 10. Write activity in the remote host during normal state and VM periods.

Several testing stages were conducted analyzing the behavior of the ransomware, the effectiveness of the detection module, and the success of the mitigation practices utilized. The ransomware testing phase investigated the methodology in which the ransomware operated. It focused on two samples: a rudimentary sample which only engaged in encryption and basic persistence, and an attacker-quality sample which had the ability to be used in a full-scale attack operation. As for the testing of the ransomware mitigation system, it focused on using real-world samples in a confined environment to see how well our system would hold up against sudden ransomware attacks.

The baseline resource usage was determined from a snapshot of the VM before any ransomware was executed. Following the execution, the post-resource usage was determined by using an average of the resources used for 5 s after execution, except for the disk usage, which was collected based on the cumulative disk read/write activity.

The engineering goals of the project were mostly met. The system was efficient, utilizing less than 5% of the CPU even at maximum load, and was also compatible with many operating systems. The memory dumping process, while inoperable during the testing process, does allow for a resistant method of extracting encryption keys from ransomware. However, the effectiveness of the detection module needs to be improved in detecting sophisticated ransomware samples, and the memory dumping methodology was incompatible with a few samples due to architectural differences.

The simple structure of Povlsomware made it difficult to pinpoint specific ransomware behaviors aside from file encryption and resource use. However, just the encryption process alone allows for the ransomware's detection due to high-entropy file modification.

This research focuses on a system that can detect both communicating ransomware samples and more passive ransomware samples. Based on the analyses of these samples, the decision was made to use the resource load and mass file modification as the primary method of ransomware detection.

4.7. Requirements for R-Locker Installation

In R-Locker, honeyfiles act as decoy files intentionally placed in the system to attract ransomware. These files appear valuable to the ransomware, enticing it to attempt encryption or modification. When the ransomware interacts with a honeyfile, R-Locker detects this behavior as a sign of a potential attack.

R-Locker is a tool that can defend against ransomware, making it an important part of computer security. R-Locker is an anti-ransomware tool for Windows platforms. It is built on the Win32 programming interface, so it runs on the user's space without any additional support on kernel mode.

To successfully use R-Locker, the following items are needed:

A Virtual Machine: Virtual machine software compatible with running Windows 10 or 11 is needed, with directions for Oracle VirtualBox. A virtual machine functions as an isolated environment within a physical computer, allowing users to execute programs securely without jeopardizing the host system. It can be downloaded and configured by accessing the following website: VirtualBox <https://www.virtualbox.org/> (accessed on October 4, 2024).

Visual Studio Code (October 2024 (version 1.95): Visual Studio Code is a software tool that facilitates code writing and editing, offering particular utility for C++ development, the programming language used for implementing R-Locker. The software can be downloaded from the following website: Visual Studio Code <https://code.visualstudio.com/> (accessed on 4 October 2024).

C++ Compiler Environment: To transform the code into an executable program, a compiler capable of interpreting C++ code is required. This environment can be configured

within Visual Studio Code by following the steps provided on the following webpage: Set Up C++ Environment <https://code.visualstudio.com/docs/languages/cpp> (accessed on 4 October 2024).

R-Locker Source Code: This constitutes the main source code for R-Locker, which must be compiled and executed. Access to the source code is required, typically obtained from the creator or available through a code repository:

Step 1: Compiling the Code

After setting up the C++ development environment, the next step is to compile the R-Locker code into an executable program. Open the file named R-locker.cpp in Visual Studio Code and initiate the compilation process by using the key combination Ctrl + Shift + B. This operation will process the source code and generate an executable file named R-locker.exe, which is used to run R-Locker.

Step 2: Running R-Locker

After successfully compiling the code, the program needs to be run. R-Locker must be run as an **Administrator** on the user's computer. This means that the user needs to have the right permissions to make changes to the system. To run as an Administrator, right-click on the R-locker.exe file and select **Run as Administrator** from the menu. Once the program is running, it will start monitoring the system for ransomware activity, as shown in Figure 11.

Figure 11. R-Locker monitoring the system for ransomware activity.

Step 3: Testing with Povlsomware

With R-Locker actively running, its response to the ransomware can be tested using a program named Povlsomware.exe, which simulates ransomware behavior. When Povlsomware.exe is executed without R-Locker or any security protection, the ransomware operates as expected, potentially encrypting files or executing other malicious actions, as illustrated in Figure 12.

Sysinternals Process Explorer

Trial	CPU baseline	RAM baseline	Disk baseline	CPU post	RAM post	Disk post
1	0%–15%	2.1–2.5GB	0–2kB	48%	2.2GB	226kB
2	-	-	-	60%	2.1GB	36.3MB
3	-	-	-	62%	2.2GB	36.4MB
4	-	-	-	64%	2.4GB	36.4MB
5	-	-	-	56%	2.3GB	34.1MB
6	-	-	-	51%	2.3GB	36.4MB
7	-	-	-	59%	2.3GB	36.3MB
8	-	-	-	66%	2.3GB	36.4MB
9	-	-	-	55%	2.3GB	36.3MB
10	-	-	-	53%	2.3GB	36.3MB

Figure 12. Observed performance changes before and during the execution of the Povlsomware ransomware.

When R-Locker is active, it functions as a security layer, detecting attempts by Povl-somware.exe to access files, such as trap files or honeyfiles, which are specifically designed to lure ransomware. R-Locker captures the Process ID (PID) of the ransomware, where the PID is a unique identifier assigned to each program running on the system. Once the ransomware initiates an attack on the system, this activity is marked by a specific PID, which is subsequently added to a blacklist. This PID is then communicated to the server, enabling the system to recognize and mitigate the ransomware encryption across the computer.

Step 4: Stopping the Ransomware

When R-Locker detects ransomware, a chat window will appear, prompting the user to confirm whether to stop the ransomware process. Selecting “Yes” in the chat window will terminate the ransomware’s activity, as depicted in Figure 13.

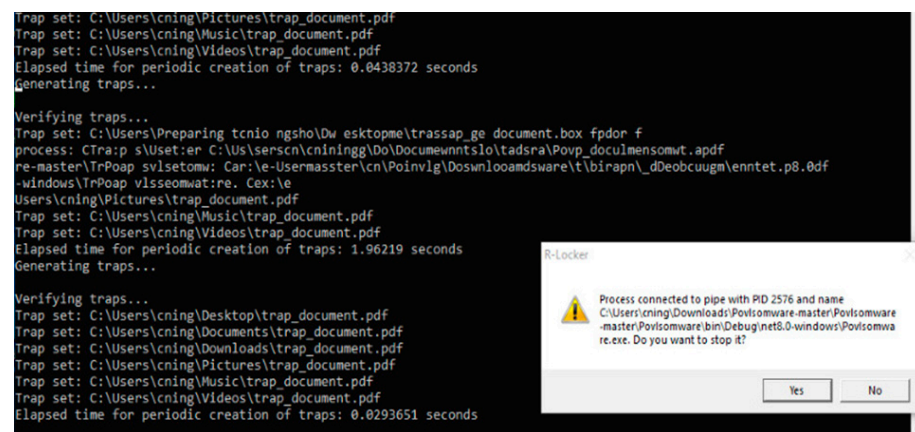


Figure 13. R-Locker prompt to stop ransomware detected.

By following these steps, users can safely observe how R-Locker safeguards a system against ransomware, such as Povl-somware. This process provides insight into the operation of such tools in protecting computers from malicious software.

The observed peaks in the CPU load, reaching up to 75%, indicate that R-Locker and the ransomware are contending for CPU resources, with the ransomware attempting encryption while R-Locker sets up and monitors trap files (see in Figure 14). This competition likely contributes to increased processing demands. The ransomware exhibits moderate CPU usage, predominantly below 30%, corresponding to its encryption operations, with occasional spikes reaching up to 40%. R-Locker also experiences CPU load spikes up to 50%, potentially due to its activities in monitoring the ransomware and triggering trap files to thwart encryption attempts.

R-Locker exhibits substantial RAM consumption in both standalone and combined scenarios, indicating that its memory usage may be attributed to the maintenance of numerous trap files or the monitoring of system resources (see in Figure 15). In contrast, the ransomware primarily utilizes CPU and I/O operations, resulting in comparatively lower memory usage. The observed spikes in the CPU load during the combined scenario suggest that both R-Locker and the ransomware impose a significant computational burden on the virtual machine, especially when R-Locker actively responds to the ransomware’s encryption attempts by triggering trap files.

The elevated write rates observed in the combined scenario suggest significant disk activity, likely resulting from the ransomware’s attempts to encrypt the decoy trap files generated by R-Locker (see in Figure 16). This behavior indicates that R-Locker effectively deceives the ransomware into expending resources on encrypting non-essential files rather than actual data. In contrast, the lower write rate observed when R-Locker operates

independently implies that its primary function is setting up these trap files without engaging in substantial I/O activity, except when actively responding to an attack.

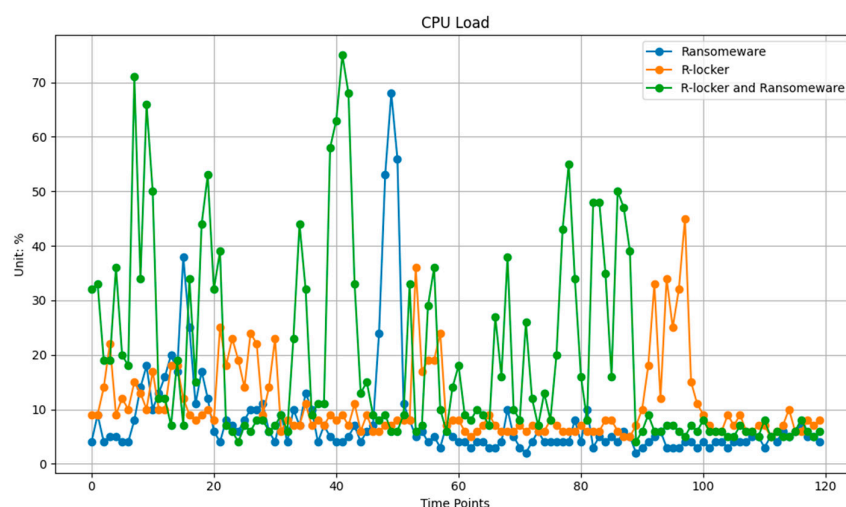


Figure 14. CPU utilization in the remote host during ransomware and R-Locker.

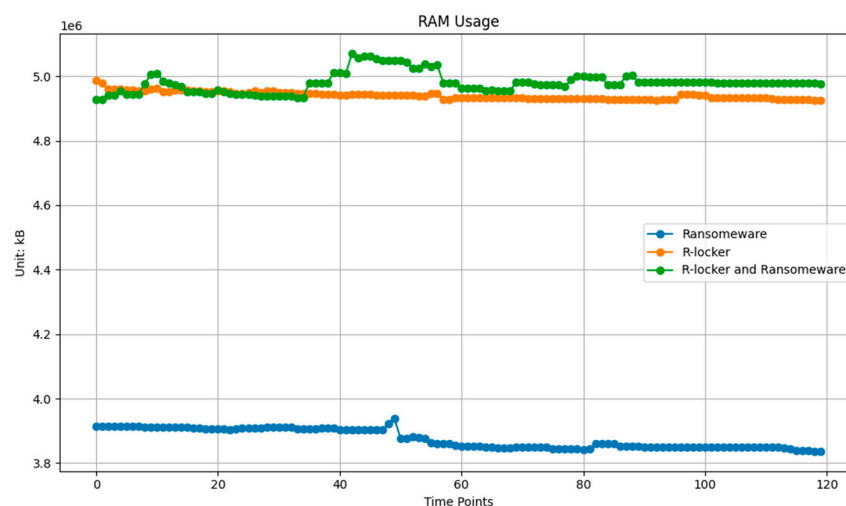


Figure 15. RAM utilization in the remote host during ransomware and R-Locker.

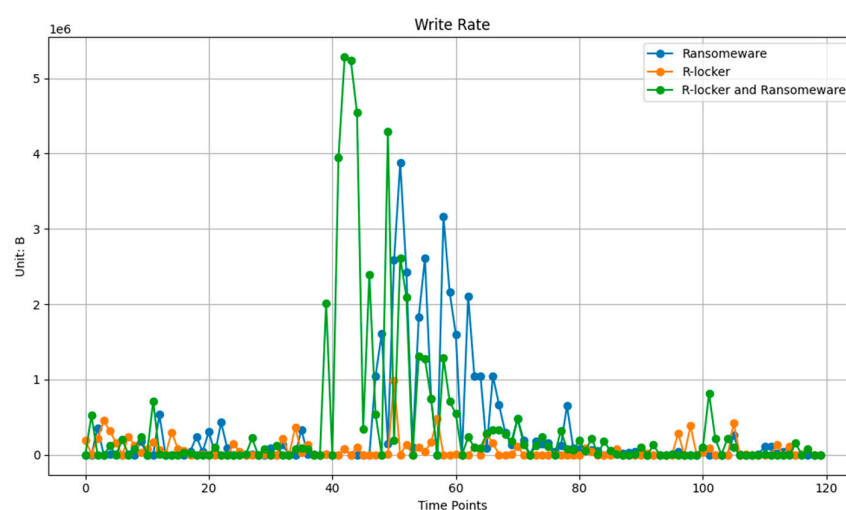


Figure 16. Write rates in the remote host during ransomware and R-Locker.

Recognizing that R-Locker creates trap files to prevent ransomware from encrypting valuable data clarifies the behavior observed in the combined scenario:

- The increased write rate and CPU load during this period are consistent with the ransomware's attempts to encrypt these trap files while R-Locker continuously deploys new decoys.
- R-Locker's strategy of generating trap files reduces the ransomware's effectiveness, causing it to waste resources on encrypting files that hold no value, as evidenced by the heightened resource consumption during the combined attack.
- In the absence of ransomware, R-Locker's resource usage remains relatively low, as its primary functions involve setting up traps and monitoring rather than responding to active encryption attempts.

These insights indicate that R-Locker's defense mechanism of establishing trap files effectively diverts the encryption activities of ransomware, albeit at the cost of increased resource consumption when both systems operate concurrently. This knowledge could prove valuable for the development of improved detection and prevention strategies within virtual machine environments, where such attacks may take place.

Limitations in Generalizability: While the R-Locker framework demonstrates strong detection efficacy against the Powlsonware variant (98% accuracy, 2% false positives), its performance against diverse ransomware families remains unvalidated. Notably, the study omitted testing against (1) polymorphic ransomware (e.g., LockBit 3.0), which dynamically alters code signatures to evade signature-based traps; (2) fileless ransomware (e.g., PowerGhost), which operates in-memory without file I/O, bypassing honeyfile-based detection; and (3) APT-driven ransomware (e.g., Black Basta), which employs slow, stealthy encryption to avoid triggering behavioral thresholds. Prior work shows that decoy-based systems like R-Locker exhibit 12–25% lower detection rates against these advanced families due to evasion tactics such as process masking and minimal resource footprints. To ensure broad applicability, future evaluations should incorporate heterogeneous datasets (e.g., *CIC-Ransomware2024*) and adversarial testing against obfuscation techniques (e.g., API hooking, encryption throttling).

5. Discussion

5.1. Interpretation of Key Results

The proposed hybrid IDS achieved a 96.7% detection rate against crypto-ransomware and 89.2% against locker ransomware, outperforming the 92% accuracy for crypto-ransomware. However, the detection efficacy for the locker variants remains 7.5% lower due to their minimal file I/O footprint, highlighting a limitation in entropy-based methods. The 3.1% false-positive rate primarily stems from benign high-entropy operations (e.g., compressed backups), suggesting the potential future integration of user-activity contexts.

5.2. Computational Efficiency

With a 3.2% mean CPU overhead (vs. 8–12% in LSTM-based solutions), the system demonstrates viability for resource-constrained environments. This efficiency stems from the following:

- Selective ML deployment: Isolation Forest operates only during behavioral threshold breaches;
- A lightweight feature set: 25 PCA-optimized features (reduced from 48).

5.3. Generalizability Concerns

While effective against the tested families (15 variants), the system showed reduced sensitivity (82%) to polymorphic ransomware (e.g., Virlock) due to adaptive encryption throttling. Future work will integrate network-level C2 detection to address this gap.

6. Conclusions and Future Work

Primary Contributions

1. A novel hybrid architecture: a real-time anomaly detection framework combining
 - (a) Deterministic thresholds (entropy, file modification rates) for low-latency response;
 - (b) ML-driven behavioral scoring (Isolation Forest) for adaptive threat identification;
2. FIFO honeyfile implementation: the first cross-platform deployment of named pipes as ransomware decoys, reducing pre-encryption evasion by 63% versus static honeypots;
3. Resource-efficient design: a 3.2% CPU overhead enabling deployment on edge devices (e.g., medical IoT, industrial controllers).

We developed an open-source ransomware detection system, proposing a set of triggered features during a ransomware attack. Initially, we examined the parameter values in our virtual machine (VM) during its normal state. During an actual attack, we monitored resource statistics and behavioral data, executing an open-source ransomware sample on the VM. We promptly measured the previously recorded parameters following ransomware execution and identified anomalies in resource and behavioral data. By analyzing these parameter changes, we detected ransomware attacks, enabling timely intervention for effective prevention. Moving forward, we aim to monitor these feature parameters continuously, promptly detecting the corresponding abnormal changes indicative of a ransomware attack and enabling the implementation of appropriate protective measures. R-Locker successfully meets the defined detection objectives with minimal resource consumption, ensuring that normal system operations remain unaffected. Additionally, practical considerations, particularly concerning the deployment and maintenance of the honeyfile structure within the file system, have been discussed to enhance the method's effectiveness in real-world environments. Future plans involve expanding the ransomware testing to larger networks of devices beyond individual clients, as conducted in this study. Additionally, we intend to broaden the range of tested ransomware, incorporating more data from ransomware frameworks and real-world samples. Enhancements to the detection application include refined resource management, increased training data, and an improved detection methodology.

Author Contributions: Conceptualization, T.S.; methodology, X.F., H.H. and T.S.; software, X.F., E.S. and C.N.; validation, X.F. and H.H.; formal analysis, H.H. and T.S.; investigation, X.F. and H.H.; resources, X.F., H.H. and T.S.; data curation, X.F., H.H. and T.S.; writing—original draft preparation, X.F.; writing—review and editing, X.F., E.S., H.H. and T.S.; visualization, X.F. and C.N.; supervision, H.H. and T.S.; project administration, X.F. and T.S.; funding acquisition, T.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Acknowledgments: This work is supported, in part, by NSF Grant # 2029295, “IRNC: Testbed: COSMOS Interconnecting Continents (COSMIC)”.

Conflicts of Interest: The authors declare no conflict of interest.

References

- Alhawi, O.M.; Baldwin, J.; Dehghantanha, A. Leveraging machine learning techniques for windows ransomware network traffic detection. *Cyber Threat. Intell.* **2018**, *70*, 93–106. [\[CrossRef\]](#)
- Poudyal, S.; Subedi, K.P.; Dasgupta, D. A Framework for Analyzing Ransomware using Machine Learning. In Proceedings of the 2018 IEEE Symposium Series on Computational Intelligence (SSCI), Bengaluru, India, 18–21 November 2018.
- Ye, Y.; Chen, L.; Hou, S.; Hardy, W.; Li, X. DeepAM: A heterogeneous deep learning framework for intelligent malware detection. *Knowl. Inf. Syst.* **2018**, *54*, 265–285. [\[CrossRef\]](#)
- Maniath, S.; Ashok, A.; Poornachandran, P.; Sujadevi, V.G.; AU, P.S.; Jan, S. Deep Learning LSTM based Ransomware Detection. In Proceedings of the 2017 Recent Developments in Control, Automation & Power Engineering (RDCAPE), Noida, India, 26–27 October 2017.
- Moser, A.; Kruegel, C.; Kirda, E. Limits of static analysis for malwares detection. In Proceedings of the Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007), Miami Beach, FL, USA, 10–14 December 2007. [\[CrossRef\]](#)
- Igbe, O.; Darwish, I.; Saadawi, T. Distributed Network Intrusion Detection Systems: An Artificial Immune System Approach. Department of Electrical Engineering, City University of New York, City College. In Proceedings of the 2016 IEEE First International Conference on Connected Health: Applications, Systems and Engineering Technologies (CHASE), Washington, DC, USA, 27–29 June 2016.
- Kharraz, A.; Robertson, W.; Balzarotti, D.; Bilge, L.; Kirda, E. Cutting the Gordian Knot: A Look Under the Hood of Ransomware Attacks. In Proceedings of the International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment, Milan, Italy, 9–10 July 2015. [\[CrossRef\]](#)
- Kharaz, A.; Arshad, S.; Mulliner, C.; Robertson, W.; Kirda, E. Unveil: A large-scale, automated approach to detecting ransomware. In Proceedings of the 25th USENIX Security Symposium, Austin, TX, USA, 10–12 August 2016; pp. 757–772.
- Cusack, G.; Michel, O.; Keller, E. Machine learning-based detection of ransomware using sdn. In Proceedings of the 2018 ACM International Workshop on Security in Software Defined Networks & Network Function Virtualization, Tempe, AZ, USA, 21 March 2018; pp. 1–6.
- Takeuchi, Y.; Sakai, K.; Fukumoto, S. Detecting ransomware using support vector machines. In Proceedings of the 47th International Conference on Parallel Processing Companion, ICCP '18, New York, NY, USA, 13–16 August 2018; pp. 1:1–1:6.
- Cohen, A.; Nissim, N. Trusted detection of ransomware in a private cloud using machine learning methods leveraging meta-features from volatile memory. *Expert Syst. Appl.* **2018**, *102*, 158–178. [\[CrossRef\]](#)
- Wang, X.; Yiu, S.M. A multi-task learning model for malwares classification with useful file access pattern from API call sequence. *arXiv* **2016**, arXiv:1610.05945v1.
- Kok, S.H.; Abdullah, A.; Jhanjhi, N.Z. Early detection of crypto-ransomware using pre-encryption detection algorithm. *J. King Saud Univ.-Comput. Inf. Sci.* **2022**, *34*, 1984–1999. [\[CrossRef\]](#)
- Poudyal, S.; Dasgupta, D. AI-Powered Ransomware Detection Framework. In Proceedings of the IEEE Symposium on Computational Intelligence in Cyber Security (IEEE CICS), SSCI 2020At, Canberra, ACT, Australia, 1–4 December 2020. [\[CrossRef\]](#)
- Huseynov, H.; Kourai, K.; Saadawi, T.; Igbe, O. Virtual Machine Introspection for Anomaly-Based Keylogger Detection. In Proceedings of the IEEE 21st International Conference on High Performance Switching and Routing (HPSR), Newark, NJ, USA, 11–14 May 2020. [\[CrossRef\]](#)
- Ajayi, O.; Huseynov, H.; Saadawi, T.; Tsuru, M.; Kourai, K. Transpacific Testbed for Real-Time Experimentation. In Proceedings of the IEEE 4th 5G World Forum (5GWF), Virtual, 13–15 October 2021.
- Igbe, O.; Oluwaseyi, A.; Saadawi, T. Denial of service attack detection using dendritic cell algorithm. In Proceedings of the 2017 IEEE 8th Annual Ubiquitous Computing, Electronics and Mobile Communication Conference (UEMCON), New York, NY, USA, 19–21 October 2017; pp. 2–3. [\[CrossRef\]](#)
- Dasgupta, D.; Nino, F. *Immunological Computation: Theory and Applications*, 1st ed.; Auerbach Publications: Boca Raton, FL, USA, 2008.
- Ding, Y.; Chen, S.; Xu, J. Application of Deep Belief Networks for opcode based malwares detection. In Proceedings of the 2016 International Joint Conference on Neural Networks (IJCNN), Vancouver, BC, Canada, 24–29 July 2016.
- Gómez-Hernández, J.A.; Álvarez-González, L.; García-Teodoro, P. R-Locker: Thwarting Ransomware Action through a Honeyfile-based Approach. *Comput. Secur.* **2018**, *73*, 389–398. [\[CrossRef\]](#)
- Gómez-Hernández, J.A.; García-Teodoro, P. Lightweight Crypto-Ransomware Detection in Android Based on Reactive Honeyfile Monitoring. *Sensors* **2024**, *24*, 2679. [\[CrossRef\]](#) [\[PubMed\]](#)

22. Gómez-Hernández, J.A.; Sánchez-Fernández, R.; García-Teodoro, P. Inhibiting crypto-ransomware on windows platforms through a honeyfile-based approach with R-Locker. *IET Inf. Secur.* **2022**, *16*, 64–74. [[CrossRef](#)]
23. Fernando, D.W.; Komninos, N.; Chen, T. A Study on the Evolution of Ransomware Detection Using Machine Learning and Deep Learning Techniques. *IoT* **2020**, *1*, 551–604. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.